First Missing Positive (hard)

Problem Statement

Given an unsorted array `nums` containing positive and negative integers, return the `smallest` `missing` positive integer value.

Note: You must implement algorithm in $o(n)$ time and uses $o(1)$ space.

Examples

- **Example 1**:
  - **Input**: [5, 3, -1, 8, 4, 2]
  - **Expected Output**: **1**
  - **Justification**: The first missing positive integer in the array is 1.
- **Example 2**:
  - **Input**: [2, 5, -7, 3, 9, 4, 6, 1]
  - **Expected Output**: **7**
  - **Justification**: The array contains all integers from 1 to 6, so the first missing positive integer is 7.
- **Example 3**:
  - **Input**: [-4, -3, -2, -1]
  - **Expected Output**: **1**
  - **Justification**: Since there are no positive integers, the first missing positive integer is 1.

Try it yourself

Try solving this question here:

Java

```java
public class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
```

```
4           // ToDO: Write Your Code Here.
5           return n + 1;
6       }
7   }
8
```

Test Case | Run Code Result | Stdout

```
[5, 3, -1, 8, 4, 2]
```

Use Example Testcases | Run | Submit

← Previous

Solution: Regular Expression Matching

Next →

Solution: First Missing Positive

Mark as Completed